

APP: Verificar Internet con CORRUTINAS (Kotlin)

1. Dependencia (IMPORTANTE)

build.gradle (Module: app)

Agrega:

```
dependencies {  
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3"  
}
```

Luego presiona **Sync Now**.

2. Permiso Internet

AndroidManifest.xml

```
<uses-permission android:name="android.permission.INTERNET"/>
```

3. Interfaz (igual que antes)

activity_main.xml

```
<LinearLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:gravity="center"  
    android:orientation="vertical">
```

```
<Button  
    android:id="@+id/btnVerificar"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Verificar Internet"/>
```

```
<TextView  
    android:id="@+id/txtEstado"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Estado"  
    android:textSize="20sp"  
    android:layout_marginTop="20dp"/>
```

```
</LinearLayout>
```

4. Kotlin con CORRUTINAS (VERSIÓN MODERNA)

MainActivity.kt

```
package com.example.pruebainternetkotlin

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import kotlinx.coroutines.*
import java.net.HttpURLConnection
import java.net.URL

class MainActivity : AppCompatActivity() {

    private lateinit var btnVerificar: Button
    private lateinit var txtEstado: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        btnVerificar = findViewById(R.id.btnVerificar)
        txtEstado = findViewById(R.id.txtEstado)

        btnVerificar.setOnClickListener {

            txtEstado.text = "Verificando..."

            CoroutineScope(Dispatchers.Main).launch {

                val resultado = withContext(Dispatchers.IO) {
                    hayInternetReal()
                }

                if (resultado) {
                    txtEstado.text = " Internet REAL disponible"
                } else {
                    txtEstado.text = " Sin conexión a Internet"
                }
            }
        }
    }
}
```

```

    }
}

private fun hayInternetReal(): Boolean {
    return try {
        val url = URL("https://clients3.google.com/generate_204")
        val conexion = url.openConnection() as HttpURLConnection
        conexion.connectTimeout = 3000
        conexion.connect()

        conexion.responseCode == 204

    } catch (e: Exception) {
        false
    }
}
}

```

¿Qué cambia aquí?

Antes usábamos:

```
Thread { }
```

Ahora usamos: `CoroutineScope(Dispatchers.Main).launch`

¿Qué significa cada parte?

Dispatchers.Main
Interfaz de usuario (pantalla)
Dispatchers.IO
Tareas pesadas (Internet, archivos, base de datos)
withContext
Cambia de hilo sin bloquear la app
Flujo del programa
Botón → Corrutina (Main) → Consulta Internet (IO) → Resultado → Actualiza pantalla (Main)
Resultado: Con Internet:
Internet REAL disponible / Sin Internet / Sin conexión a Internet